

[v2.0] 项目状况分析与分工规划

1. 项目当前实现状态分析 (Status Report)

根据代码库现状 (CourseSelectionSystem_knowledge_base.md) 和 v1.0 的开发成果，我们已经完成了“从 0 到 1”的跨越，但也引入了严重的**架构违规**风险，必须在 v3.0 阶段立刻修正。

-  **已达成成就 (What's Done)**
 - **全栈跑通**：实现了 `CLI -> Controller -> DBAdapter -> PostgreSQL` 的完整链路。
 - **基础交互**：学生选课、退课的 CLI 菜单已可用。
 - **数据落地**：数据库表结构已建立，数据可持久化。
-  **严重风险 (Critical Issues)**
 - **架构违规 (扣分点)**：目前 `SystemController` 直接调用了 `m_db.execute` (SQL语句)。
 - **需求文档强制约束**：“应用逻辑层.....不得直接访问数据库”。
 - **需求文档强制约束**：“领域层.....严禁出现SQL语句”。
 - **功能缺失**：教学秘书（排课）、教师（打分）、时间冲突检测算法尚未实现。
-  **v3.0 阶段目标**
 - **架构重构**：引入 **代理者模式 (Proxy Pattern)**，将 SQL 从 Controller 剥离到 Infrastructure 层。
 - **业务补全**：完成教学秘书和教师的核心功能。

2. 核心分工策略 (Lead vs Member)

为了在修正架构的同时推进业务，我们继续采用“核心-外围”的分工模式。

角色	高扬职责	张涛职责
定位	架构师 & 核心算法 (Architect)	业务逻辑开发 (Feature Dev)
负责层级	 基础设施重构 + 领域核心	 应用业务扩展 + 基础数据层
关键任务	1. 实现 Proxy 模式 (清洗 Controller 中的 SQL) 2. 冲突检测算法 (<code>Timeslot::overlaps</code>) 3. 解决 N+1 查询 (复杂 JOIN 查询)	1. 教学秘书模块 (创建课程业务) 2. 成绩管理模块 (录入/查看成绩) 3. 简单 Proxy (模仿 Lead 的代码实现简单的 CRUD)
难度系数	☆☆☆☆☆ (涉及架构合规性)	☆☆☆ (涉及业务广度)

3. 协作流水线 (Workflow)

1. **高扬 先行 (Define):** 高扬先创建一个标准的 `StudentProxy` 和 `Timeslot` 类，定义好“怎么把 SQL 藏在 Infrastructure 层”的规范。
2. **张涛 跟进 (Follow):** 张涛参考你的 `StudentProxy`，照猫画虎写出 `CourseProxy` (用于秘书排课) 和 `GradeProxy` (用于打分)。
3. **Controller 瘦身 (Refactor):** 最后，高扬负责把 `SystemController` 里原本乱写的 SQL 删掉，全部替换为调用张涛和高扬要写的 Proxy 接口。

4. 详细任务清单 (To-Do Lists)

● 高扬 的任务清单：架构重构与核心算法

你的核心目标是消除扣分点。

- **任务 A: 基础设施层重构 (Infrastructure)**
 - 目标: 剥离 SQL，实现 ORM 映射。
 - 文件: `src/CourseSelectionSystem/infrastructure/infra.student_proxy.cppm`
 - 内容:
 - 实现 `findById(id)`: 编写复杂的 JOIN SQL，一次性取出学生信息和已选课程。
 - 实现 `save(student)`: 将学生对象的内存状态（新选的课）更新回数据库。
- **任务 B: 领域核心算法 (Domain)**
 - 目标: 实现时间冲突检测。
 - 文件: `src/CourseSelectionSystem/domain/dom.timeslot.cppm`
 - 内容:
 - 定义 `Timeslot` 结构体 (weekday, session)。
 - 实现 `bool overlaps(const Timeslot& other)` 算法。
 - c修改 `Course` 类，使其包含 `Timeslot` 成员。
- **任务 C: 应用层清洗 (Application)**
 - 目标: 移除 `SystemController` 中的 SQL。
 - 动作: 将 `m_db.execute(...)` 全部替换为 `StudentProxy::save(...)` 或 `_courseProxy.findAll(...)`。

● 张涛 的任务清单：业务功能开发

开发任务单: CourseSelectionSystem (Phase 2)

v1.0 阶段我们已经跑通了基础流程。在 v2.0 阶段，我们需要完成**教学秘书**和**教师**的功能。

重要提示：为了符合架构要求，请不要在 Controller 中直接写 SQL。请参考高扬即将提交的 `StudentProxy` 范例，为你负责的模块编写简单的 Proxy 类。

1. 教学秘书模块 (Secretary Features)

- 目标: 实现“创建课程”功能。
- 文件: `infrastructure/infra.course_proxy.cppm` (新建)
- 代码参考:

```
1 export module course_system:infrastructure.course_proxy;
2 import :domain.course;
3 import :infrastructure; // 引用 DBAdapter
4 import std;
5
6 export class CourseProxy {
7 public:
8     // 将课程对象保存到数据库 (用于教学秘书"创建课程")
9     static bool addCourse(const Course& course) {
10         db::DBAdapter db;
11         // TODO: 请根据实际表结构完善 SQL
12         std::string sql = std::format(
13             "INSERT INTO courses (id, name, capacity, weekday, section)
14             VALUES ('{0}', '{1}', {2}, {3}, {4})",
15             course.getId(), course.getName(), course.getCapacity(), 1, 1
16         );
17         return db.execute(sql);
18     };
19 }
```

- UI 对接: 在 `showSecretaryMenu` 中接收输入, 构建 `Course` 对象, 调用上述 Proxy。

2. 成绩管理模块 (Grade Features)

- 目标: 实现教师打分和学生查分。
- 文件: `infrastructure/infra.enrollment_proxy.cppm` (新建)
- 任务:
- 实现 `updateScore(studentId, courseId, score)`: 执行 SQL `UPDATE enrollments SET score = ...`。
- 在 `TeacherMenu` 中调用此接口。

3. 提交要求

- 确保 `SystemController` 能够调用你写的 Proxy 接口。

5. 最终交付形态对比 (Before vs After)

模块	v2.0 (当前)	v3.0 (目标)
SystemController	包含大量 SQL 字符串 (违规)	只有业务逻辑, 调用 Proxy (合规)
Domain (Course)	只有 ID/Name	包含 <code>Timeslot</code> 对象, 支持 <code>overlaps()</code>
功能范围	仅学生选课	学生/教师/秘书 全流程跑通
数据访问	直接操作 DBAdapter	通过 Proxy 类间接操作

